



INSTITUTO POLITÉCNICO NACIONAL ESCUELA SUPERIOR DE CÓMPUTO

PRÁCTICA 1

USO DE GALAXY

DISEÑO DE SISTEMAS DIGITALES

4CM4

INTEGRANTES:

MONTEALEGRE ROSALES DAVID URIEL
SALAUZ HERNANDEZ NERICK FRANCISCO
VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

AGUILAR SANCHEZ FERNANDO

INSTITUTO POLITÉCNICO NACIONAL



FECHA DE ENTREGA: 13 DE OCTUBRE DE 2024

1. OBJETIVO

Al finalizar la práctica, el alumno aprenderá a programar un dispositivo lógico programable (PLD). Para esto estudiará y aprenderá a utilizar las herramientas adecuadas para lograr dicho objetivo. Así mismo, reafirmará el conocimiento adquirido en clase al realizar un programa que ejecute una función específica, sobre el dispositivo utilizado, y comprobar físicamente el correcto funcionamiento del programa desarrollado. Finalmente, sabrá lo que es un PLD y como programar una función específica sobre el dispositivo, utilizando un lenguaje de descripción de hardware (HDL).

II OBJETIVOS ESPECÍFICOS:

- I. Distinguir los beneficios proporcionados por las herramientas CAD.
- II. Confirmar el conocimiento adquirido en clase y realizar un programa en un HDL que se ejecute sobre un PLD.
- III. Comprobar físicamente el correcto funcionamiento del sistema diseñado, el cual será implementando sobre un protoboard.

III MATERIAL Y EQUIPO EMPLEADO

- ☐ Mesa de instrumentación, del laboratorio de sistemas digitales ☐ Programador universal
- ☐ Fuente de 5V
- ☐ 1 GAL22V10
- ☐ 1 DIP switch de 4
- ☐ 3 Resistencias de $1K\Omega$
- ☐ 7 Resistencias de 220Ω
- ☐ 1 Display de Ánodo común ☐

IV INTRODUCCIÓN TEÓRICA:

En esta práctica, se exploraron las funcionalidades de Galaxy, una plataforma integral de diseño de sistemas digitales. El objetivo principal fue aprender a utilizar sus herramientas para el desarrollo y simulación de circuitos digitales. Galaxy es una suite que permite a los diseñadores implementar, simular y verificar el funcionamiento de sistemas basados en hardware, facilitando tanto el desarrollo de prototipos como la validación de diseños.

Durante el ejercicio, se abordaron conceptos fundamentales relacionados con la lógica digital. Se trabajó con diseños combinacionales, integrando componentes básicos para desarrollar sistemas más complejos. Galaxy proporcionó una interfaz gráfica que permitió la creación y simulación de estos sistemas, lo que ayudó a los estudiantes a

visualizar el comportamiento del hardware antes de proceder a su implementación física.

El proceso incluyó la introducción al entorno de trabajo de Galaxy, la construcción de esquemas digitales y la ejecución de simulaciones para verificar la correcta operación del sistema diseñado. A través de esta práctica, los estudiantes adquirieron conocimientos clave sobre cómo utilizar herramientas automatizadas de diseño, las cuales son esenciales en la industria moderna de desarrollo de hardware digital.

4.1 LAS HERRAMIENTAS CAD- EDA

(Computer-Aided Design y Electronic Design Automation) son software especializados que facilitan el diseño, simulación, y verificación de sistemas electrónicos. Estas herramientas permiten a los ingenieros y diseñadores desarrollar circuitos integrados y sistemas digitales de manera eficiente y precisa. Dentro de este campo, las herramientas CAD ayudan en la creación del diseño físico, mientras que las EDA automatizan tareas complejas del proceso de diseño.

Entre las funciones principales de estas herramientas están la creación de esquemas, la simulación del comportamiento lógico y eléctrico de los circuitos, el análisis de potencia y rendimiento, así como la generación de los archivos necesarios para la fabricación de circuitos integrados. Ejemplos populares de herramientas CAD-EDA incluyen **Xilinx Vivado**, **Altium Designer**, y **Cadence Virtuoso**, que son ampliamente utilizados en la industria para el diseño de FPGAs, ASICs, y PCBs. Estas herramientas mejoran la eficiencia del proceso de diseño y ayudan a evitar errores costosos en la etapa de fabricación.

4.2 Lenguajes de descripción de hardware (HDL)

Los **Lenguajes de Descripción de Hardware (HDL)** son lenguajes de programación utilizados para modelar, diseñar y describir el comportamiento y la estructura de sistemas digitales a nivel de hardware. A diferencia de los lenguajes de programación tradicionales que describen el flujo de ejecución de un software, los HDL permiten la especificación del funcionamiento de circuitos electrónicos, como puertas lógicas, flip-flops, registros, y otros componentes digitales.

Existen dos lenguajes HDL ampliamente utilizados en la industria:

- **VHDL (VHSIC Hardware Description Language):** Un lenguaje robusto y versátil que permite describir circuitos a nivel de comportamiento y estructura. Es ampliamente usado en sistemas de misión crítica, como los sistemas aeroespaciales y de defensa, debido a su fuerte tipado y sintaxis estricta.
- **Verilog:** Es un lenguaje más simple y directo en comparación con VHDL. Es popular en la industria de diseño de circuitos integrados y es preferido para la modelación de sistemas más comerciales debido a su enfoque orientado a la síntesis y simulación eficiente.

Los HDL permiten describir el comportamiento del sistema a diferentes niveles de abstracción (como a nivel de compuertas o de comportamiento) y, mediante herramientas de síntesis, convertir estas descripciones en circuitos físicos que pueden ser implementados en FPGAs o ASICs. Esto facilita el proceso de diseño y verificación, permitiendo la simulación de diseños complejos antes de su fabricación.

4.3 Los dispositivos lógicos programables (PLD's)

Los Dispositivos Lógicos Programables (PLD's) son circuitos integrados cuya funcionalidad lógica puede ser configurada por el usuario después de su fabricación, permitiendo una gran flexibilidad en el diseño de sistemas digitales. A diferencia de los circuitos integrados con una función fija (como puertas lógicas o flip-flops individuales), los PLD's pueden ser programados para realizar diversas tareas, lo que los convierte en una solución versátil para desarrollar y probar hardware sin necesidad de diseñar un circuito a medida desde cero.

Existen diferentes tipos de PLD's, siendo los más comunes:

- CPLD (Complex Programmable Logic Device): Un tipo de dispositivo lógico programable que ofrece una capacidad de lógica moderada y está formado por bloques lógicos interconectados. Es adecuado para aplicaciones que requieren una lógica compleja, pero no tanto como un FPGA, y es más fácil de programar y menos costoso.
- FPGA (Field-Programmable Gate Array): Son los dispositivos más avanzados y ofrecen un número mucho mayor de recursos lógicos. Están compuestos por miles de bloques lógicos que pueden ser interconectados para realizar cualquier función digital compleja. Los FPGA son utilizados en aplicaciones de alta demanda de rendimiento y flexibilidad, como en telecomunicaciones, procesamiento de señales, y sistemas embebidos.

Estos dispositivos son programados utilizando Lenguajes de Descripción de Hardware (HDL) como VHDL o Verilog, y pueden ser reconfigurados cuantas veces sea necesario, permitiendo modificaciones en el diseño sin tener que fabricar nuevos circuitos físicos. Los PLD's juegan un rol crucial en el prototipado rápido, el desarrollo de productos y la investigación en diseño digital, ya que combinan la flexibilidad de programación con la eficiencia y velocidad de un sistema hardware.

4.3.1 La GAL 22V10

La GAL 22V10 (Generic Array Logic) es un dispositivo lógico programable que cuenta con una arquitectura flexible y versátil, que permite implementar funciones tanto combinacionales como secuenciales. A continuación se detalla su arquitectura y características internas basadas en el manual del dispositivo:

Arquitectura del GAL 22V10

1. Matriz AND-OR Programable:

- El dispositivo utiliza una matriz lógica programable compuesta por un conjunto de puertas AND y OR. La matriz permite a los usuarios personalizar el diseño para implementar diferentes combinaciones lógicas y generar múltiples funciones booleanas.
- Las entradas del dispositivo se conectan a una red de puertas AND, lo que permite crear múltiples términos de producto que luego se combinan mediante puertas OR para generar las salidas.

2. Entradas y salidas:

- El GAL 22V10 tiene 22 pines, que pueden configurarse como entradas, salidas o bidireccionales, dependiendo de la configuración programada por el usuario. Estas entradas y salidas se conectan directamente a la matriz lógica, lo que brinda flexibilidad al dispositivo.
- De los 22 pines, 12 están reservados como entradas dedicadas, mientras que los otros 10 pueden configurarse como entradas o salidas, dependiendo de la configuración de las macroceldas.

3. Macroceldas:

- El dispositivo contiene 10 macroceldas configurables, cada una de las cuales puede operar en modo combinacional o secuencial (registrado). Las macroceldas son bloques lógicos que permiten al usuario definir el comportamiento de las salidas, ya sea como salidas combinacionales o como salidas basadas en registros.
- Las macroceldas incluyen un flip-flop tipo D, que puede activarse para almacenar estados en aplicaciones secuenciales, como contadores o registros.
- El flip-flop puede configurarse para operar en modo síncronico (con reloj) o asíncronico (con funciones de preset y clear), proporcionando flexibilidad para aplicaciones secuenciales.

4. Registros:

- Los flip-flops tipo D en cada macrocelda son cruciales para el almacenamiento y procesamiento de datos en aplicaciones secuenciales. Los registros se sincronizan con una señal de reloj externa y pueden retener un valor lógico hasta que se actualice con una nueva entrada o condición de control.
- El dispositivo incluye controles adicionales para gestionar el comportamiento de los registros, como las señales de preset (configuración a 1) y clear (limpieza a 0), que permiten inicializar o resetear las salidas.

5. Señal de Reloj:

- El GAL 22V10 tiene una entrada de reloj dedicada que sincroniza todas las macroceldas cuando se configuran en modo secuencial. Esto permite que los flip-flops cambien de estado en base a la señal de reloj, lo que es crucial para aplicaciones como contadores o máquinas de estados.

6. Feedback interno:

- El dispositivo permite el feedback interno, lo que significa que las salidas de las macroceldas pueden retroalimentarse a la matriz de puertas AND. Esto es útil para crear funciones lógicas más complejas o para implementar circuitos secuenciales, como contadores y máquinas de estado.

Modos de Operación de las Macroceldas

El GAL 22V10 puede ser programado en varios modos dependiendo de las necesidades del diseño:

- Modo Combinacional: Las macroceldas funcionan como simples funciones lógicas combinacionales, sin el uso de flip-flops. Las salidas dependen únicamente de las entradas en tiempo real.
- Modo Registrado: Las macroceldas emplean el flip-flop para almacenar un estado y generar salidas secuenciales, lo que es útil en diseños que requieren temporización o memoria, como contadores y registros.

Memoria y Programación

- El GAL 22V10 tiene una matriz EEPROM interna que permite programar las configuraciones de las puertas AND y OR, así como las macroceldas. Esto significa que puede reprogramarse múltiples veces, facilitando el desarrollo y la prueba de diferentes diseños sin tener que desechar el dispositivo.

Características Adicionales

- Protección contra sobreescritura: El GAL 22V10 incluye un mecanismo de seguridad para evitar la reprogramación accidental del dispositivo una vez que ha sido programado y está en uso.
- Configuración de salida: Las salidas pueden configurarse como salidas de tres estados, lo que permite a las macroceldas interactuar con otros componentes en un bus de datos compartido.

En resumen, el GAL 22V10 es un dispositivo flexible y robusto, con una arquitectura programable que lo hace ideal para la implementación de funciones lógicas personalizadas, desde simples combinaciones hasta diseños más complejos con lógica secuencial.

4.4 Herramientas de desarrollo

Las herramientas de desarrollo son fundamentales en el diseño y la programación de sistemas digitales, ya que permiten a los ingenieros diseñar, simular, verificar y programar dispositivos

lógicos programables como FPGAs, CPLDs, y GALs. Estas herramientas incluyen tanto software como hardware, cada una diseñada para facilitar diferentes aspectos del desarrollo. A continuación se presentan algunas de las herramientas más utilizadas en la industria y en entornos educativos.

Herramientas de Software

1. ISE Design Suite (Xilinx):

- ISE es una suite de desarrollo proporcionada por Xilinx que permite el diseño de sistemas digitales para FPGAs y CPLDs. Es una herramienta integral que incluye funcionalidades para la síntesis de HDL (Verilog y VHDL), simulación, análisis de tiempos y diseño de circuitos lógicos.
- La suite incluye una interfaz gráfica que facilita la creación de proyectos, la verificación funcional y la programación de dispositivos Xilinx, además de generar los archivos de bitstream necesarios para configurar los FPGAs.
- Aunque ha sido reemplazada en parte por Vivado para los dispositivos más recientes, sigue siendo utilizada para dispositivos más antiguos como la serie Spartan.

2. Quartus II (Intel/Altera):

- Quartus II, desarrollado originalmente por Altera (ahora propiedad de Intel), es una herramienta poderosa para el diseño de sistemas digitales en dispositivos FPGA y CPLD de Intel. Al igual que ISE, permite la síntesis, simulación y análisis de circuitos lógicos basados en HDL, y tiene un flujo de trabajo optimizado para la creación de proyectos FPGA.
- Quartus II también incluye un potente editor de esquemáticos para aquellos que prefieren un enfoque gráfico al diseño. Es compatible con dispositivos populares como las familias Cyclone y Stratix de Intel.
- Una de sus características clave es el soporte para diseños basados en procesadores embebidos y sistemas multiprocesador, lo que lo hace ideal para aplicaciones avanzadas.

3. Galaxy (Cypress Semiconductor Corporation):

- Galaxy es una suite de desarrollo proporcionada por Cypress Semiconductor que está especialmente diseñada para trabajar con dispositivos CPLD y otras soluciones lógicas programables de la marca.
- Incluye herramientas de síntesis de HDL y simulación, así como soporte para la programación de sus dispositivos lógicos. Es una herramienta flexible y adaptable a diferentes proyectos de diseño digital, aunque su uso ha sido más limitado comparado con ISE o Quartus II.

Herramientas de Programación (Hardware)

1. Programador Universal:

- En entornos educativos y profesionales, los programadores universales son dispositivos que permiten programar una amplia variedad de circuitos lógicos programables, desde GALs hasta FPGAs y CPLDs. Estos programadores son muy versátiles y son capaces de configurar una gran diversidad de dispositivos mediante diferentes protocolos de programación.
- Estos dispositivos se conectan a una computadora y se comunican con las herramientas de desarrollo mediante interfaces estándar, como USB o serial. En el ámbito educativo, los programadores universales son esenciales para enseñar conceptos de diseño digital y verificación en hardware real.

2. USB Blaster (Intel/Altera):

- El USB Blaster es una herramienta de programación de hardware utilizada para cargar bitstreams en dispositivos FPGA y CPLD de Intel/Altera. Es un programador económico y ampliamente utilizado que se conecta a través de un puerto USB y permite la configuración rápida de los dispositivos desde el software Quartus II.
- Es muy popular tanto en entornos educativos como en la industria debido a su simplicidad y confiabilidad.

3. Xilinx Platform Cable USB:

- Similar al USB Blaster de Altera, el Xilinx Platform Cable USB es un dispositivo de programación utilizado para cargar configuraciones en dispositivos FPGA y CPLD de Xilinx. Se conecta mediante USB a una computadora y se utiliza junto con ISE o Vivado para programar FPGAs.
- Este programador es compatible con una amplia gama de dispositivos Xilinx y permite una programación rápida y eficiente.

4. Cypress Programmer:

- El Cypress Programmer es una herramienta de hardware utilizada para programar los dispositivos CPLD y otras soluciones lógicas de Cypress Semiconductor. Proporciona una interfaz sencilla para cargar configuraciones en dispositivos lógicos, y es compatible con la suite Galaxy.
- Aunque menos común en comparación con los programadores de Xilinx o Intel, es útil para aquellos que trabajan específicamente con la tecnología de Cypress.

DESARROLLO

5.1 Ambiente de desarrollo

El ambiente de desarrollo utilizado en esta práctica es Galaxy v. 6.3IR35 de Cypress Semiconductor Corporation. Este entorno de desarrollo integrado (IDE) está diseñado específicamente para el diseño, simulación y programación de dispositivos lógicos programables como las GAL (Generic Array Logic) y CPLD (Complex Programmable Logic Device).

Galaxy incluye varias herramientas que facilitan el proceso de diseño:

- **Editor de Esquemáticos y HDL:** Permite a los diseñadores crear circuitos lógicos tanto utilizando lenguajes de descripción de hardware (HDL) como Verilog y VHDL, o mediante diagramas esquemáticos gráficos.
- **Simulación:** Proporciona un simulador que permite verificar la funcionalidad del diseño antes de programarlo en el hardware, asegurando que el comportamiento lógico sea correcto.
- **Síntesis:** Convierte el diseño lógico en un archivo de configuración que puede ser cargado en el dispositivo programable.
- **Programación:** Incluye una herramienta de programación que facilita la carga del diseño en dispositivos como la GAL 22V10, utilizando programadores compatibles con Cypress.

El ambiente Galaxy es eficiente y optimizado para diseñadores que trabajan con dispositivos de lógica programable, ofreciendo una interfaz intuitiva para el desarrollo y verificación de diseños digitales.

5.2 Problema

El problema propuesto es simple, pues la finalidad es relacionarse con las herramientas CAD, comprender la mecánica del proceso de diseño utilizando PLD's y realizar la implementación física de la función deseada. De esta manera, el problema consiste en:

Programar e implementar físicamente, verificando el correcto funcionamiento, de un decodificador (DEC). El DEC tiene tres entradas y ocho salidas. Las salidas serán capaz de mostrar, en un display de 7 segmentos, el dato decodificado que recibe en la entrada.

Nota: El DEC deberá mostrar en su salida un número de boleta, por ejemplo: 89560171

Para el diseño se deberá programar utilizando uno de los siguientes métodos: método de Ecuaciones ó When else ó With select when ó If then ó Case when, del lenguaje VHDL a través del IDE Galaxy. El DEC deberá mostrar en su salida un número de boleta, por ejemplo: 89560171

5.3 Solución del problema

Entradas			Segmentos del Display (Salidas)							Número (Boleta)
E3	E2	E1	A	B	C	D	E	F	G	
0	0	0	0	0	0	0	0	0	0	8
0	0	1	0	0	0	1	1	0	0	9
0	1	0	0	1	0	0	1	0	0	5
0	1	1	0	1	0	0	0	0	0	6
1	0	0	0	0	0	0	0	0	1	0
1	0	1	1	0	0	1	1	1	1	1
1	1	0	0	0	0	1	1	1	1	7
1	1	1	1	0	0	1	1	1	1	1

```

LIBRARY IEEE;
USE IEEE.STD_LOGIC_1164.ALL;
ENTITY DECO IS
PORT (
E : IN STD_LOGIC_VECTOR( 2 DOWNTO 0 );
DISPLAY : OUT STD_LOGIC_VECTOR( 6 DOWNTO 0 )
);
END DECO;
ARCHITECTURE ADECO OF DECO IS
BEGIN
PDECO : PROCESS( E )
BEGIN
CASE E IS
WHEN "000" => DISPLAY <= "0000000"; -- 8
WHEN "001" => DISPLAY <= "0001100"; -- 9
WHEN "010" => DISPLAY <= "0100100"; -- 5
WHEN "011" => DISPLAY <= "0100000"; -- 6
WHEN "100" => DISPLAY <= "0000001"; -- 0
WHEN "101" => DISPLAY <= "1001111"; -- 1
WHEN "110" => DISPLAY <= "0001111"; -- 7
WHEN OTHERS => DISPLAY <= "1001111"; -- 1
END CASE;
END PROCESS PDECO;
END ADECO;

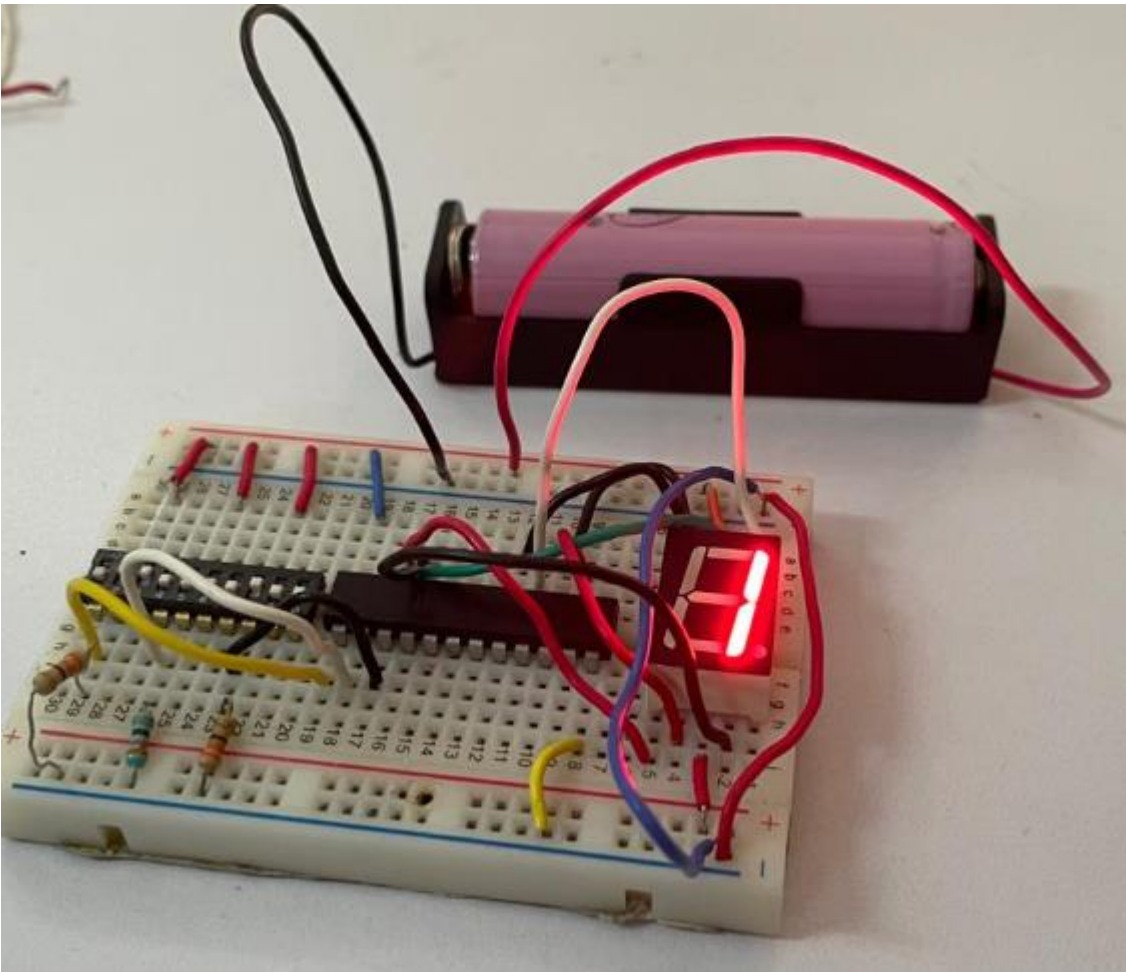
```

```

C22V10

```

e(2) = 1	24 * not used
e(1) = 2	23 = display(2)
e(0) = 3	22 = display(6)
not used * 4	21 = display(4)
not used * 5	20 * not used
not used * 6	19 * not used
not used * 7	18 * not used
not used * 8	17 = display(0)
not used * 9	16 = display(5)
not used * 10	15 = display(1)
not used * 11	14 = display(3)
not used * 12	13 * not used



VI OBSERVACIONES Y CONCLUSIONES

Montealegre Rosales David Uriel

El desarrollo del decodificador en VHDL mostró la flexibilidad y el control que ofrece este lenguaje para describir el comportamiento del hardware. La implementación de estructuras como CASE facilitó la organización y claridad del código, permitiendo un diseño modular y fácil de entender.

La familiarización con VHDL es fundamental para diseñadores de sistemas digitales, ya que permite una descripción precisa del comportamiento de circuitos lógicos. Utilizar Galaxy para desarrollar y simular VHDL no solo mejora la comprensión del lenguaje, sino que también optimiza el proceso de diseño y validación de sistemas digitales complejos.

Vázquez Blancas Cesar Said

El uso del ambiente Galaxy v. 6.3IR35 demostró la importancia de seguir un flujo de diseño estructurado al desarrollar el decodificador de 7 segmentos. La transición desde la descripción en VHDL hasta la síntesis y simulación permite identificar y corregir errores lógicos antes de implementar el diseño en hardware. La interfaz de Galaxy facilita este proceso, aunque se requiere atención para evitar errores en la escritura del código.

La herramienta Galaxy es esencial para el diseño digital, ya que integra de manera efectiva las etapas de descripción, simulación y síntesis. Seguir este flujo correctamente es crucial para garantizar que el diseño funcione según lo esperado y evitar problemas durante la implementación en hardware.

Salauz Hernández Nerick Francisco

La práctica con Galaxy v. 6.3IR35 para diseñar el decodificador de 7 segmentos resaltó la importancia de comprender tanto la teoría como la práctica en el diseño digital. A través de esta experiencia, pude apreciar cómo la correcta aplicación de conceptos teóricos de lógica digital se traduce en un diseño efectivo en VHDL. Además, el uso de la herramienta de simulación me permitió validar rápidamente el comportamiento del circuito, lo que confirma que el diseño es funcional antes de su implementación en hardware. Esta práctica no solo fortaleció mi comprensión del flujo de trabajo en herramientas CAD-EDA, sino que también me motivó a seguir explorando el desarrollo de sistemas digitales complejos en el futuro.

5. BIBLIOGRAFIA

Cypress Semiconductor Corporation. *GAL 22V10 Programmable Logic Device Data Sheet*. [Online] Available: <https://www.cypress.com/documentation/datasheets/gal-22v10-programmable-logic-device-data-sheet>

□ **Zainalabedin Navabi.** *VHDL: Analysis and Modeling of Digital Systems*. 3rd ed. New York: McGraw-Hill, 2008.

□ **P. H. Bardell, W. H. McAnally, y J. D. Savir.** *Digital Circuit Testing and Debugging*. 2nd ed. New York: Wiley-Interscience, 2004.